AIPL: Actor based Intelligent Parallel Language

— 設計から実現 —

ABCL/1 の継承と、型・効果・期限による静的保証

児玉靖司

法政大学経営学部

yass@hosei.ac.jp

2026 年 8 月 21 日

概要

本稿は、アクターを単位とする型付き並行言語 AIPL（Actor based Intelligent Parallel Language、処理系名 ABCLc+）の設計と実装を、一つの論文としてまとめたものである。AIPL は ABCL/1（Yonezawa, Briot, Shibayama, OOPSLA 1986）を出発点とし、その三種のメッセージ送信（past / now / future 型）をほぼそのまま受け継ぐ。一方で ABCL/1 が持っていた express mode による割り込み、返信先の一級性、`script` によるパターン受信は、意図的に落とすか大幅に縮小した。代わりに AIPL が得たのは静的な保証である。すなわち **型**（`reply` の引数から戻り値型を推論し、注釈も書ける）、**効果**（メソッドが世界に及ぼす作用を 8 種の名前で宣言・照合する）、**期限**（`now`/`await` に上限時間を義務づける）の三つであり、この三つは互いに噛み合っている。たとえば「実時間で動くアクターが機外の言語モデルを同期で呼ぶ」ことは、効果検査だけで禁止できる。

形式的な保証は二段になっている。機械検証が済んでいるのは効果と期限を持たない中核 AIPL^{-} であり、保存・進行・型安全性・メッセージ不理解の不在を Rocq (Coq) で検証した。本稿の執筆にあたり主要 6 定理について `Print Assumptions` を実行し、いずれも公理に依存しないこと（`Closed under the global context`）を確認している。効果と期限を加えた AIPL^{-2} は**紙の上の証明**であり、そこでは期限を構文に義務づけた結果として進行定理の「待ちでブロックする」枝が消え、系としてデッドロック自由が言える。この範囲の機械検証は未了で、必要な追加は三点に特定できている。

実現は三つの処理系からなる。正となる OCaml 版（手書き 7,289 行）、Python の CLI 実装（14,443 行）、ブラウザで動く JavaScript 実装（5,709 行）である。三実装はガイドのサンプル 9 本のうち 8 本で出力が完全に一致した。残る 1 本の差は言語仕様の差ではなく OCaml 版 REPL の出力取りこぼしであり、20 回反復して 13 回は全行が揃うという形で定量化した。

最後に、実測によって確認できた穴を二つ報告し、**三実装とも直した**。`now` を `future` と `await` に分けて書くと効果検査を逃れること、および異なるクラスのアクター型が単一化してしまうことである。どちらも最小例で再現したうえで修正し、手元の `.aipl` 543 本を修正前後に通して、判定が変わったのが**狙ったテストだけ**であることを確かめた。併せて「弾かれるべきプログラム」を資産として持つ仕組みを作ったところ、出力の比較では見えなかった実装間の食い違いが五つ出た —- 効果伝播の穴は正であるはずの OCaml 版だけのものであり、JS-I は効果を集めていながら宣言と照合しておらず、Py-I はトップレベルの呼び出しの引数を一つも検査していなかった。さらに探りを入れて、三実装すべてに共通する穴を一つ見つけた —- **未宣言の名前への代入が黙って通る**。フィールド名を打ち間違えると、フィールドは更新されないまま別の変数ができ、何のエラーも出ない。これも三実装とも直した。弾かれるべきプログラムは 8 本から 15 本に増え、期待と食い違う枠は 45 枠中 9 枠、OCaml 版は 15 本すべてを弾く。最後に、小林直樹の型システム研究の系譜が、AIPL が未解決として残した設計課題にどう噛み合うかを整理する。

目次

1 はじめに 4

2 背景 5
- 2.1 ABCL/1 . . . 5
- 2.2 型理論的な基礎づけ——小林の系譜 . . . 5
- 2.3 ABCL/f . . . 6

3 AIPL の設計 6
- 3.1 三つの設計指針 . . . 6
- 3.2 構文 . . . 6
- 3.3 三種の送信 . . . 7
- 3.4 `reply` と `sender` . . . 8
- 3.5 `reply` からの戻り値型推論 . . . 8
- 3.6 型 . . . 8
- 3.7 効果 . . . 8
- 3.8 期限 . . . 9
- 3.9 その他の機構 . . . 9
- 3.10 ABCL/1 から落としたもの . . . 10

4 形式的な保証 10
- 4.1 証明が成立する範囲を切り出す . . . 10
- 4.2 主定理 . . . 10
- 4.3 紙の上の証明 — AIPL^{-2} . . . 11
- 4.4 機械検証に残っている作業 . . . 11
- 4.5 機械検証の状態 . . . 12

5 実現 12
- 5.1 三つの処理系 . . . 12
- 5.2 OCaml 版の構成 . . . 13
- 5.3 三実装を揃え続ける . . . 13

6 評価 13
- 6.1 型検査が捕まえるもの . . . 13
- 6.2 三実装の一致 . . . 14
- 6.3 出力の非決定性を定量化する . . . 14
- 6.4 応用 . . . 15

7 現状の限界 15
- 7.1 効果伝播の穴——`future`+`await` が検査を逃れる（修正済み） . . . 16
- 7.2 アクター型が区別されない（修正済み） . . . 17

7.3 弾かれるべきプログラムを資産にする 17
7.4 三実装を並べて初めて見えたもの 18
7.5 その他 19

8 **小林研究をどう取り込むか** 19

9 **関連研究** 20

10 **おわりに** 21

1　はじめに

AIPL は二つの目的のために作られている。一つは**分散 AI エージェント**の記述である。複数の言語モデルや外部サービスを、それぞれ独立に動くアクターとして書き、その協調をプログラムとして残したい。もう一つは**実時間 OS** の記述である。Xinu のような小型 OS の構成要素——プロセス管理、メールボックス、スケジューラ、デバイスドライバ——を、C の手続き群ではなく型付きアクター群として書きたい。

この二つは一見すると遠い。前者は「機外のモデルを呼ぶので、いつ返ってくるか分からない」世界であり、後者は「期限を守れないことが故障である」世界である。しかし両方ともアクターとメッセージで書きたいという点では同じであり、そして**両者を混ぜてはいけない**という点でも同じである。実時間で動くアクターが、機外のモデルを同期で待ってはならない。この「混ぜてはいけない」を、規約ではなく型検査で禁止したい——これが AIPL の設計を貫く動機である。

出発点として ABCL/1 を選んだ理由は二つある。第一に、ABCL/1 は三種のメッセージ送信（past / now / future）を言語の中核に据えた数少ない設計であり、この三種はそのまま「待たない / 待つ / 後で受け取る」という AIPL が必要とする区別に対応する。第二に、ABCL/1 には型がない。動的型の並行オブジェクト言語である。ここに静的な型・効果・期限を載せると何が言えるようになるかは、それ自体が問いとして立つ。

本稿の貢献

1. ABCL/1 から何を継承し、何を落としたかを、計算モデルの水準で整理した（§2.1、§3.10）。落とした三つ——express mode、返信先の一級性、`script` のパターン受信——について、落とした理由を型システムと証明の側から述べる。
2. 型・効果・期限という三つの静的な仕組みを設計し、それらが互いに噛み合う形を示した（§3）。とくに `reply` の引数から戻り値型を推論する仕組みは、ABCL/1 が持たなかった「返信に型を付ける」問題への一つの答である。
3. 中核 AIPL^{-} の保存・進行・型安全性を機械検証し、効果と期限を加えた AIPL^{-2} を紙の上で証明した（§4）。期限を構文に義務づけると進行定理の枝が一本減り、系としてデッドロック自由が言えることを示す。両者の境目と、機械検証に残っている三点を明示する。
4. 三つの処理系による実現と、それらを揃え続けるための仕掛けを述べた（§5）。
5. 実測にもとづく評価と、現状の限界を報告した（§6、§7）。限界は「未実装項目の列挙」ではなく、最小例で再現した穴である。二つの穴は三実装とも修正し、既存 543 本への影響を実測した。併せて「弾かれるべきプログラム」の集合を資産として整備した。
6. 小林直樹の型システム研究の系譜が、これらの穴にどう噛み合うかを整理した（§8）。

本稿の位置づけ

AIPL については、これまで個別の主題ごとにレポートを書いてきた。ABCL/1 との比較 [18]、落とした機能を戻した場合の検討 [19]、型健全性の第 1 版 [20] と第 2 版 [21]、`reply` からの戻り値型推論 [22]、ユーザーズガイド [23]、三実装の同期 [24]、小林研究の取り込み検討 [25] である。本稿はそれらを

前提とせずに読める形で一つにまとめ直したものであり、各主題の詳細は該当レポートに譲る。実測値は本稿の執筆時点（2026 年 8 月 21 日、対象コミット `c837eef`）で取り直した。

2 背景

2.1 ABCL/1

ABCL/1[1, 2] は、並行オブジェクトを計算の単位とする言語である。オブジェクトは自分のメールボックスを持ち、到着したメッセージを一つずつ処理する。この「一度に一つ」がオブジェクト内部の逐次性を保証し、それゆえフィールドへの排他制御を書かなくてよい。AIPL もこの性質をそのまま引き継いでいる。

ABCL/1 の中核は次の四つである。

■三種のメッセージ送信。 *past* 型は送って待たない。*now* 型は送って返信を待つ。*future* 型は送って引換券を受け取り、後で中身を取り出す。この三分割が ABCL/1 の最も影響力のある設計であり、AIPL はこれを `send` / `now` / `future`+`await` としてほぼそのまま継承した。

■`script` によるパターン受信。 オブジェクトは受け取るメッセージのパターンを並べ、さらに `where` 節で状態依存の受理条件を書ける。「在庫があるときだけ購入メッセージを受け取る」といった記述が、メソッド本体の `if` ではなく受理の段階で書ける。

■express mode による割り込み。 ordinary モードとは別の待ち行列があり、express メッセージは実行中の処理を中断して割り込む。中止・優先処理・監視のための機構である。

■返信先の一級性。 *now* 型の送信で暗黙に作られる返信先（reply destination）は値であり、他のオブジェクトへ渡せる。つまり「A が受けた依頼に、B が代わりに答える」という委譲が書ける。

2.2 型理論的な基礎づけ——小林の系譜

ABCL/1 は動的型であり、型システムを持たない。その基礎づけを与える試みは早くから行われた。小林と米澤の一連の研究 [8, 7, 6] は、線形論理を土台として並行オブジェクトの型理論的基礎を与えた。ここで重要なのは、後に AIPL が直面する問題の多くがすでにこの系譜の中で扱われていることである。

- **線形型** [9]——「ちょうど一度だけ使う」を型で保証する。返信先を一級の値にしたときに「返信が一度だけ行われる」を保証するのは、まさにこの問題である。
- **デッドロック自由な型システム** [10, 11, 12]——通信の依存関係に順序を入れ、循環待ちを型で排除する。
- **π 計算の汎用型システム** [13]——個別の型システムを作り分けるのではなく、一つの枠組みから複数の解析を導く。
- **資源使用解析** [14]——資源の使い方（開いてから閉じる、等）を型で追う。効果を順序を持つ列として扱う立場である。
- **高階モデル検査** [15, 16]——交差型による型付けとモデル検査の等価性。検証器を自作せずに借りるための道具立てである。
- **精製型・サイズ型** [17]——計算量や応答時間の上界を型で述べる。

AIPL が現在抱えている課題——返信先を一級にしたいが「ちょうど一度」を守りたい、待ちの循環を防ぎたい、効果に順序を入れたい、期限の上界を型で述べたい——は、この一覧とほぼ一対一に対応する。§8 でこの対応を具体的に述べる。

2.3 ABCL/f

同じ ABCL 系で型を持つ先行例として ABCL/f[3] がある。future を中核に据えた多相型付き並行オブジェクト言語であり、「三種の送信のうち future だけを残して型を付ける」という設計判断をとった。AIPL は三種すべてを残したまま型を付けようとしている点で異なる。この違いが型システムに何をもたらすかは §3.3 で述べる。

3 AIPL の設計

3.1 三つの設計指針

■(1) アクターが単位である。 プログラムはクラス宣言の並びと、トップレベルの文からなる。クラスはフィールドとメソッドを持ち、`new` でアクターの実体になる。アクターは自分のメールボックスを持ち、メッセージを一つずつ処理する。

■(2) 値は `return` ではなく `reply` で返す。 メソッドは値を返すのではなく、送り主に向けて `reply(e)` で返信する。これは ABCL/1 の返信の考え方をそのまま受けたものだが、静的型を載せるうえでは厄介な選択でもある。`return` なら「関数の型」が戻り値型を決めるが、`reply` は本体のどこにでも書ける文だからである。この厄介さへの答が §3.5 の戻り値型推論である。

■(3) 静的な仕組みは三つだけにする。 型・効果・期限の三つに絞り、それぞれを独立に理解できる形にした。そのうえで、三つが噛み合うことによって初めて言えることを設計の中心に置いた。

3.2 構文

現行実装（`src/parser.mly`、229 行・27 規則・18 トークン）の文法を要約する。

```
program    ::= decl+

decl       ::= "class" ID "{" field* method+ "}"
             | "var" ID "=" expr ";"
             | "var" ID "=" "new" ID "(" args ")" ";"
             | "send"  target "." ID "(" args ")" ";"
             | ID "(" args ")" ";"

field      ::= "var"   ID "=" expr ";"
             | "float" ID "=" expr ";"

method     ::= "method" ID "(" params ")" ret? eff? "{" stmt+ "}"
params     ::= param ("," param)*
param      ::= ID | ID ":" type            // 引数の型注釈
ret        ::= ":" type
eff        ::= "!" "{" (ID ("," ID)*)? "}"
type       ::= "int"|"float"|"string"|"bool"|"unit"|"any"|ClassName

target     ::= ID | "remote" "(" STRING "," STRING ")"
```

```
stmt       ::= ID "=" expr ";"
             | "var" ID "=" expr ";"
             | "var" ID "=" "new" ID "(" args ")" ";"
             | ID "(" args ")" ";"  |  "call" ID "(" args ")" ";"
             | "send" ("self"|"sender"|target) "." ID "(" args ")" ";"
             | "if" "(" expr ")" stmt ("else" stmt)?
             | "while" expr "do" stmt
             | "{" stmt* "}"
             | "become" ID "(" args ")" ";"
             | "select" "{" case* timeout_case? "}"

case         ::= "case" ID "(" ids? ")" "->" "{" stmt+ "}"
timeout_case ::= "timeout" INT "->" "{" stmt+ "}"

expr       ::= INT | FLOAT | STRING | "true" | "false" | ID
             | "-" expr
             | expr ("=="|"!="|">"|"<"|">="|"<="|"++"|"+"|"-"|"*"|"/") expr
             | "new" ID "(" args ")"
             | "now" target "." ID "(" args ")" ("timeout" INT "else" expr)?
             | "future" target "." ID "(" args ")"
             | "await" expr ("timeout" INT "else" expr)?
             | ID "(" args ")"  |  "(" expr ")"
```

演算子の優先順位は次のとおりで、下にいくほど強く結合する。

弱	`==`　`!=`
↓	`>`　`<`　`>=`　`<=`
	`++`（文字列連結）
	`+`　`-`
強	`*`　`/`
	単項マイナス

二項演算子は**すべて左結合**で、等値も例外ではない。`1 == 2 == false` は `((1 == 2) == false)` と読まれて `true` になる。

注意 1. `++` が文字列連結で、`+` は数値専用である。この分離は後から入れたもので、移行のときに一つ罠を踏んだ。`+` は `++` より強く結合するため、`"a" + b + c` の `+` を一つだけ `++` に替えると意味が変わる。正規表現での一括置換ではなく、構文木で連鎖を判定して置換する道具を書いた。

3.3　三種の送信

書き方	ABCL/1	意味	式か文か
`send t.m(a);`	past	送って待たない	文
`now t.m(a) timeout` n `else` e	now	送って返信を待つ	式
`future t.m(a)` / `await f ...`	future	引換券を受け取る	式

型を付けるうえで、この三種は同じではない。`send` は返信を受け取らないので、メソッドの戻り値型を要求しない。`now` は式なので、その場に戻り値型が要る。`future` は τ を返すメソッドに対して `future`$\langle\tau\rangle$ を返し、`await` がそれを τ に戻す。ABCL/f が future だけを残したのは、この三番目だけ

を扱えば型付けが素直に済むからである。AIPL は三種を残したまま、send には戻り値型を要求しないという扱いで整理した。

3.4 reply と sender

メソッドの返信は reply(e) で行う。送り主は sender で参照でき、send sender.m(...) と書ける。ここで型検査が見るのは次の二点である。

- **被覆**：戻り値型が宣言されているとき、すべての実行経路で reply に到達しなければならない。
- **線形性**：一つのメッセージに対する reply は**高々一度**である。

select の case 本体にある reply は、囲むメソッドではなく case が受理したメッセージに帰属する。この帰属を間違えると被覆判定が壊れるため、実装では両者を別に数えている。

3.5 reply からの戻り値型推論

戻り値型の注釈 method m(x) : T は**省略できる**。省略された場合、reply の引数から型を推論する。実装ではメソッドごとに戻り値型を表す型変数 ρ を用意し、本体に現れる各 reply(e) について ρ と e の型を単一化する。

推論だけでは決まらない場面が 6 つあることを別稿 [22] で整理した。代表的なものは、すべての経路で reply しないメソッド、経路によって異なる型を返すメソッド、相互再帰するアクター群である。そこで実装は「推論する。ただし注釈も書ける」という形をとった。注釈があれば、それが照合先となって「経路によって型が違う」「そもそも返していない」を具体的に指摘できる。

注意 2. ρ を型スキームに含めてはならない。含めると呼び出しごとに ρ が新しくなり、「このメソッドはどの型を返すか」という制約が呼び出し側へ伝わらなくなる。ρ はクラスのメソッド表に属する量であって、多相化の対象ではない。

3.6 型

型言語は次のとおりである（src/types.ml）。

```
τ ::= int | float | string | bool | unit | any
    | τ list -> τ                  (関数)
    | actor[C]{ m : τ, ... }       (アクター)
    | array<τ> | future<τ> | { l : τ, ... }
    | α                            (型変数)
σ ::= ∀α□…α□. τ                        (型スキーム)
```

推論は Hindley–Milner 風である。クラスのメソッド表を先に走査して各メソッドの型スキームを登録し（preinfer_all_classes）、そのうえで本体を推論する。この二段構えにしないと、相互に now し合うアクターの型が決まらない。

3.7 効果

メソッドは自分が世界に及ぼす作用を宣言できる。

```
method plan(g) : string !{ai, net} { reply(ai_call("plan " ++ g)); }
```

効果は 8 種類の名前からなる固定の語彙である。

`mut`	自分の状態を変える	`net`	ネットワークに出る
`time`	時間に依存する	`ai`	言語モデルを呼ぶ
`io`	入出力	`fs`	ファイルを触る
`mem`	動的に割り付ける	`log`	ログを出す

注釈を書かなければ本体から推論する。書いた場合は本体から推論した集合と照合し、足りなければ型エラーとする。効果を生む構文は実装上明確で、フィールドへの代入と `become` が `mut`、`new` が `mem`、`remote(…)` 宛の送信が `net`、組込み関数はそれぞれ固有の効果を持つ。

■これで何が禁止できるか。 実時間側のアクターに `!{time, log}` と書いておけば、そのメソッドから `ai` や `net` を持つメソッドを同期で呼ぶことが型エラーになる。「実時間の処理が言語モデルの応答を待つ」という設計上の誤りが、規約ではなく型検査で止まる。これが §1 で述べた動機に対する直接の答である。

3.8 期限

`now` と `await` には期限を書ける。

```
var r = now p.plan("x") timeout 100 else "";
var v = await f timeout 50 else 0;
```

期限を書かない `now`/`await` は、既定では警告になる。

```
[warn] line 3, col 7: now without a deadline waits forever;
       write `now ... timeout <ms> else <expr>`
```

環境変数 `AIOS_STRICT_DEADLINE=1` を立てると、これは型エラーに昇格する。このスイッチは単なる厳しさの調整ではない。**型健全性の定理が成立する範囲に留まるためのスイッチ**である（§4）。

3.9 その他の機構

■`select`。 複数のメッセージを待ち受け、来たものから処理する。`timeout` 節を書ける。ABCL/1 の `script` のパターン受信を縮小したもので、パターンはメソッド名と引数の並びに限られ、`where` 節は無い。

■`become`。 自分の振る舞いを別のクラスに置き換える。アクターモデルの `become` をそのまま持つ。状態を変えるので `mut` 効果を生む。

■境界。 `remote("host:port", "actor")` で他ホストのアクターを指せる。また Web 経由で外部に公開する機構がある。どちらも `net` 効果を伴う。

3.10 ABCL/1 から落としたもの

ABCL/1 の機構	AIPL	落とした理由
express mode	無し	逐次性の証明が壊れる
返信先の一級性	無し	「ちょうど一度」を型で守れない
`script` の `where`	無し（`select` に縮小）	進行定理に枝が増える

三つとも「入れられない」のではなく「入れると証明の形が変わる」という理由で落としている。別稿 [19] で `where` と express mode を戻した場合を検討した。結論を要約すると、どちらも**型システムはほとんど壊さない**。どちらも「いつメッセージを受理するか」の話であって「どんな型を持つか」の話ではないからである。壊れるのは進行定理の側で、`where` は「どのガードも成立せず受理できない」という枝を一本増やし、express mode は「メソッド本体の実行が逐次である」という前提を破る。

返信先の一級性については事情が異なる。これは型の話であり、「返信先を値として渡せる」なら「渡された先が返信を一度だけ行う」を保証しなければならない。線形型がそのための道具であり、§8 で述べる。

4 形式的な保証

4.1 証明が成立する範囲を切り出す

現行実装のすべてに対して型健全性を主張することはできない。そこで、実装から証明が成立する範囲 AIPL^{-2} を切り出した [21]。判断の基準は「実装の事実」であり、どの機能が入り、どれが外れ、なぜそうなるかを一つずつ決めている。

入るもの：クラス表とメソッド表、アクターの状態ストア、メールボックスと非同期送信、future 表と `await`、動的なアクター生成、効果、期限付きの待ち。外れるもの：`become`、`select` の一部、外部公開、組込みの一部。

4.2 主定理

機械検証が済んでいるのは AIPL^{-} **—- 効果も期限も持たない中核である。**`coq/AIPLSoundness.v`（1,121 行）に次が含まれる。形式化は変数・局所束縛・代入補題、アクターの状態ストアとその型不変条件、クラス表とメソッド表、メールボックスと非同期送信、future 表と `await`、動的なアクター生成を含む。アクター型はクラス番号で添字づけられており（`TActor : nat -> ty`）、異なるクラスは異なる型である。

定理	内容
`preservation`	型付けされた構成の簡約は型付けを保つ
`preservation_star`	上記の反射推移閉包
`progress`	型付けされた構成は終状態か簡約できる
`type_safety`	型付けされた構成から到達できる構成は行き詰まらない
`no_method_not_understood`	「メソッドが無い」実行時エラーは起きない
`state_type_invariant`	アクター状態の型不変条件
`future_type_invariant`	future 表の型不変条件
`no_await_no_deadlock`	`await` を含まない構成はブロックしない
`async_deadlock_free`	`await` を含まないプログラムのデッドロック自由

定理 3 (進行、機械検証済み)**.** AIPL^{-} の型付けされた構成 C は、終状態であるか、簡約できるか、**未解決 *future* の `await` でブロックしている**かのいずれかである。

機械検証されている進行定理は三択であり、第三の枝がデッドロックである。`async_deadlock_free` はこの第三の枝を消した主張ではない。仮定に `conf_afree`（構成が `await` を含まない）が要る、すなわち**完全に非同期なプログラムに限った**デッドロック自由である。

系 4 (機械検証済み)**.** *`await`* を含まない型付き構成は、どこまで実行してもブロックせず、終状態でなければ必ず一歩進める。

4.3 紙の上の証明 —- AIPL^{-2}

効果と期限を加えた AIPL^{-2} については、別稿 [21] が**紙の上で**証明している。そこでの主結果は**進行定理が二択になる**ことである。AIPL^{-2} には期限のない待ちが構文に存在せず、待ちは残り時間を減らす簡約として進むので、ブロックの枝が消える。系として、`await` を含む場合も含めてデッドロック自由が言える。これは第 1 版 [20] が「一般には言えない」と明記した性質であり、期限を構文に義務づけたことの見返りである。代償も明確で、**停止性は依然として言えない**。期限が保証するのは「待ちが有限時間で終わる」ことであって、プログラム全体が停止することではない。さらに、デッドロックが消えるのではなく**有限時間の失敗に変換される**という点も正しく述べておく必要がある。相互に待ち合う二者は両方が期限切れになる。詰まらないが正しくもない。

副産物として、効果の健全性から「実時間側のアクターがエージェント側を待てない」ことが命題として出る。§3 で述べた設計上の要請が回収される形である。

4.4 機械検証に残っている作業

AIPL^{-2} を機械検証するために必要な追加は、別稿 [21] が三点に特定している。

1. **効果を型と対にして持つ**。判断を $\Gamma \vdash e : \tau \,!\, \varepsilon$ にし、構成の整合条件に効果条件を足す。効果集合は有限なので有限集合で足りる。
2. **期限を `await` に持たせる**。時間を進める簡約と期限切れの簡約を加える。進行定理の `await` の場合はむしろ易しくなる（三つの規則が網羅的なので、ブロックの場合分けが消える）。
3. **future 型を `future`$\langle \tau \,!\, \varepsilon \rangle$ に変える**。future 表が対を持つようになる。

三点目は §7 で報告する実装の穴と**同じもの**である。モデルの側も現状は `TFut : ty -> ty` であって効果を持たない。つまりこの一点は、実装とモデルの両方で同時に直すべき箇所である。

4.5 機械検証の状態

本稿の執筆にあたり、証明の状態を実際に確認した（2026 年 8 月 21 日）。以下はすべて AIPL⁻ についての値である。

項目	実測
処理系	The Rocq Prover 9.1.0（OCaml 5.1.1 でビルド）
形式化の規模	`coq/` 全 6 ファイル 2,757 行
	うち `AIPLSoundness.v` 1,121 行、`AIPLDining.v` 566 行
再コンパイル	`AIPLSoundness.v` が 0.9 秒で通り、生成物は既存とバイト一致
公理への依存	主要 6 定理すべて `Closed under the global context`

最後の行が重要である。`Print Assumptions` を `preservation`、`progress`、`type_safety`、`no_method_not_understood`、`no_await_no_deadlock`、`async_deadlock_free` の 6 つに対して実行し、いずれも公理・未証明の補題に依存しないことを確認した。形式化には `Axiom` も `Admitted` も含まれない。

注意 5. `coq/` には他に、ABCM（ABCL カーネル言語の意味論）の形式化と、そこから AIPL への埋め込みが翻訳の健全性つきで入っている（`ABCM.v`、`ABCMEmbedding.v`）。また食事する哲学者の Chandy–Misra 解法の形式化（`AIPLDining.v`）がある。本稿では扱わない。

5 実現

5.1 三つの処理系

AIPL には実装が三つある。

実装	位置づけ	規模	実行の形
OCaml 版	**正**	手書き 7,289 行 （生成 3,670 行）	ネイティブの REPL
Py-I（Python）	追随	14,443 行	CLI
JS-I（JavaScript）	追随	5,709 行	ブラウザ（CLI 足場あり）

OCaml 版が仕様の正である。生成 3,670 行は `ocamllex`/`ocamlyacc` が `lexer.mll`（106 行）と `parser.mly`（229 行）から作るものである。Py-I の行数はテストを除いた値で、テストを含めると 16,048 行になる。

三つも作る理由は二つある。第一に、JS-I はブラウザで動くので、処理系を配らずに言語を試してもらえる。第二に、**三つ揃えること自体が仕様の検査になる**。一つの実装しかなければ、実装の癖と言語の仕様が区別できない。

5.2 OCaml 版の構成

ファイル	行	役割
`lexer.mll`	106	字句
`parser.mly`	229	構文（27 規則・18 トークン）
`ast.ml`	426	抽象構文木
`types.ml`	468	型・型スキーム・効果の語彙・クラスのメソッド表
`typing_env.ml`	285	型環境・組込みの型と効果
`infer.ml`	1,270	型推論・効果検査・`reply` 検査・期限検査
`eval_thread.ml`	2,291	評価器（スレッドによるアクター実行）

型検査の実体は `infer.ml` に集まっている。推論と効果と `reply` 検査を別々のパスにせず、一度の走査で行っているためである。

5.3 三実装を揃え続ける

仕様を変えるたびに三つが揃っているかを確かめる必要がある。そのために、同じサンプルを三実装で流して出力を並べる道具を用意した（`tools/run_all_impls.sh`）。何かを触ったら必ずこれを流す。過去に二度、これで退行に気づいた。

揃える過程で 6 つの不具合が出た記録は別稿 [24] にある。そこで得た教訓を一つだけ引く。**検査の指摘は件数ではなく実物を開いて確かめる**。`reply` 検査が 62 件の指摘を出したとき、そのうち 59 件は誤検出であった（Py-I のメソッドは `return` でも返せる、`function` は検査対象外）。件数だけを見ていれば「大量の不具合がある」と誤診していた。

6 評価

以下はすべて 2026 年 8 月 21 日に実測した値である。

6.1 型検査が捕まえるもの

本稿のために最小の誤りプログラムを 4 本作り、OCaml 版に通した結果を示す。

■(1) `reply` の被覆漏れ。

```
class C {
  method f(x) : int {
    if (x > 0) { reply(1); }        // else の経路で reply しない
  }
}
```

```
[Type error] line 2, col 10: method f is declared to reply with int,
             but some execution path does not reply
```

■(2) `reply` の線形性違反。

```
class C { method f(x) : int { reply(1); reply(2); } }
```

```
[Type error] line 2, col 10: method f may reply more than once on some path;
             a method must reply at most once
```

■(3) 期限のない待ち。 既定では警告、AIOS_STRICT_DEADLINE=1 で型エラーに昇格する。

```
[Type error] line 3, col 7: now without a deadline waits forever;
             write `now ... timeout <ms> else <expr>`
```

■(4) 効果注釈と本体の食い違い。

```
class C {
  var n = 0;
  method f(x) : int !{log} { n = n + 1; reply(n); }   // mut を書いていない
}
```

```
[Type error] line 3, col 10: method C.f declares {log} but its body has {mut}
             (missing {mut})
```

四つとも、指摘の文面が**何を書けばよいか**まで述べている。期限の警告が `timeout <ms> else <expr>` という書き方を示すのは、この検査が「禁止」ではなく「書き換えの要求」だからである。

6.2 三実装の一致

ガイドのサンプル 9 本（g1–g9）を三実装で流した。

サンプル	内容	三実装の出力	判定
g1_hello	クラス・send・文字列連結	hello,AIPL/tick1/tick2	一致
g2_now_future	now/future/await	twice=43/awaited=20/v=7	一致
g3_actors	複数アクター	ok,left=7/soldout	一致
g4_select	select と reply の帰属	waiting/got1/viaselect:1	一致
g6_effects	効果注釈	1/1/n=1	一致
g7_deadline	期限	fast=42/slow(timedout)=0/await=10	一致
g8_bool_equality	真偽値と等値	literaltrueok/…/b=true	一致
g9_actor_arg	アクターを引数に渡す	（後述）	差あり

8 本が完全に一致した。9 本目の g9 だけが食い違ったが、これは**言語仕様の差ではない**。Py-I と JS-I が `got actor / after send / pong` の 3 行を出すのに対し、OCaml 版がその回に限って先頭行を落としたためである。

6.3 出力の非決定性を定量化する

g9 を OCaml 版で 20 回反復した。

結果	回数
3 行すべて揃った	13
行が欠けた	7
時間切れ	0

他のサンプルについても同様に反復した。

サンプル	期待行数	全行が揃った回数
`g1_hello`	3	5/10
`g2_now_future`	3	10/10
`g3_actors`	2	10/10
`g4_select`	3	10/10
`g6_effects`	3	9/10
`g7_deadline`	3	6/6
`g8_bool_equality`	4	5/6
`g9_actor_arg`	3	13/20

所見 6. 出力の欠落は、待ちを含むサンプル（`g2`、`g3`、`g4`、`g7`）では一度も起きず、`send` だけで終わるサンプル（`g1`、`g9`）で頻繁に起きる。これは計算が誤るのではなく、REPL がスクリプトの終端に達したときにまだ書き出されていない出力を捨てているためだと考えられる。`now`/`await` を含むサンプルでは待ちの間に書き出しが進むので、問題が表に出ない。

この所見は、**単発の実行では絶対に得られない**。一度流して 3 行出れば「動いている」と判断してしまう。反復して分布を見て初めて、「動くこともある」と「常に動く」の区別がつく。

6.4 応用

AIPL は言語単体ではなく、いくつかの応用の記述言語として使われている。

- **災害時の避難支援 MANET**[26]。基地局が失われた状況で、避難者の端末そのものを無線ノードとし、その上を移動エージェントが渡り歩く方式を、AIPL のアクターと Xinu 互換の軽量ノードで実装した。1 台の端末を 1 つのノードとして独立プロセス化している。
- **小型 OS の記述** [27]。Xinu 風 OS の構成要素——プロセス管理、メールボックス、スケジューラ、デバイスドライバ、システムコール——をアクターに対応づける構想。システムコール引数、デバイスの権限、プロセス間メッセージを型で検査する設計である。
- **進化計算によるアクター構成の探索** [28]。アクター組織の設計パラメータを遺伝子とし、同一の進化エンジンを 4 つの業務ドメインに適用した。

これらは AIPL の設計判断が現場で妥当かを試す場でもある。とくに効果と期限は、実時間側（OS・MANET）とエージェント側（言語モデル）を同じプログラムに書いたときに初めて意味を持つ。

7 現状の限界

未実装項目を並べるのではなく、本稿のために最小例を作って**実際に再現した**穴を二つ報告する。

7.1 効果伝播の穴——future+await が検査を逃れる（修正済み）

§6 の (4) で見たとおり、効果注釈は本体と照合される。ところが、同じ意味の処理を now で書くか future+await で書くかで結果が変わる。

```
class Planner {
  method plan(g) : string !{ai, net} { reply(ai_call("plan " ++ g)); }
}
class Ctrl {
  var p = new Planner();
  method viaNow() : unit !{time, log} {          // ai も net も宣言していない
    var r = now p.plan("x") timeout 100 else "";
    print(r);
  }
  method viaFutureAwait() : unit !{time, log} {  // 同じ意味
    var f = future p.plan("x");
    var r = await f timeout 100 else "";
    print(r);
  }
}
```

viaNow だけを含むプログラムは、期待どおり弾かれる。

```
[Type error] line 7, col 10: method Ctrl.viaNow declares {log, time}
             but its body has {ai, log, net} (missing {ai, net})
```

ところが viaFutureAwait だけを含むプログラムは、**通ってしまう**。

```
[Loaded] /tmp/aipl/e5_gap.aipl
```

これは深刻である。§3 で述べた「実時間側が言語モデルを待てない」という保証が、書き方を変えるだけで破れることを意味する。

原因は型の定義にあった。future$\langle\tau\rangle$ が値の型 τ しか持たず、効果を持たない。そのため future を作った時点で効果が落ち、await は何も足さない。

■直した。 TFuture of ty を TFuture of ty * SSet.t ref に変え、future が呼び先（"**クラス名#メソッド名**"）を型に載せ、await がその呼び先へ向けて now と同じ効果伝播の辺を張るようにした。集合にしてあるのは、別々の呼び出し由来の future が単一化されうるためで、単一化のときは両側の和をとって書き戻す。これで now 版と future+await 版が同じ診断を出す。

```
[Type error] line 8, col 10: method Ctrl.viaFutureAwait declares {log, time}
             but its body has {ai, log, net} (missing {ai, net})
```

■既存コードへの影響。 手元の 4 リポジトリにある .aipl **543 本**を修正前後の型検査に通し、判定が変わったファイルを数えた。変わったのは**狙ったテスト 1 本だけ**である（修正前 OK 136 / 構文エラー 268 / 型エラー 139、修正後 OK 135 / 268 / 140）。構文エラーが多いのは、他実装向けの方言で書かれた .aipl が OCaml 版の構文では読めないためで、修正前後で変わらない。

7.2 アクター型が区別されない（修正済み）

型言語には actor[C] があるにもかかわらず、異なるクラスのアクター型が単一化してしまう。

```
class Alpha { method a() : unit !{log} { print("a"); } }
class Beta  { method b() : unit !{log} { print("b"); } }
class Use {
  method f(x: Alpha) : unit !{log} { send x.a(); }   // Alpha を要求している
}
var u = new Use();
var b = new Beta();
send u.f(b);          // ところが Beta を渡す
```

修正前はこれが通った。Beta に a は無いので、受け付けてはならない。単一化が TActor 同士を無条件に成功させていたことが原因である。第 2 版の健全性レポートで「モデルが実装より厳しい」と書いた箇所が、まさにこれであった。形式化ではアクター型がクラス番号で添字づけられている（TActor : nat -> ty）で no_method_not_understood が定理として立つが、実装はその前提を満たしていなかった。

■直した。 単一化をクラス名で行うようにした。ただし実行時の値から作られるアクター型はクラスが静的に分からず TActor ("?", []) になるので、"?" はどちらとも合う印として扱う。

```
[Type error] line 11, col 1: actor type mismatch: actor(Alpha) vs actor(Beta)
```

逃げ道として AIOS_LAX_ACTOR=1 で従来の挙動に戻せるようにしたが、543 本を通した結果、判定が変わったのは**やはり狙ったテスト 1 本だけ**で、逃げ道を使う必要は無かった。

注意 7. この二つの穴には共通の性質がある。どちらも**正しいプログラムでは表に出ない**。型検査は通り、実行も正常に進む。誤ったプログラムを書いたときにだけ、「止めるはずのものが止まらない」という形で現れる。だから通常の動作確認では見つからず、定式化のために仕様を書き下す過程で初めて露見した。形式化には「証明する」以外に「実装の穴を見つける」という効用がある。そして**見つけたあとも、通常のテストでは守れない**。守るには「弾かれるべきプログラム」を資産として持つ必要がある（§7.3）。

7.3 弾かれるべきプログラムを資産にする

穴を直すだけでは足りない。既存の突き合わせ（§5）は**通るプログラムの出力が一致するか**しか見ておらず、**通ってはいけないプログラムが止まるか**を見る仕組みが無かった。§6 で示した誤りプログラムも、その場限りのものだった。

そこで、弾かれるべきプログラムを 8 本並べた集合と、それを三実装すべてに通す道具を用意した（docs/samples/reject/、tools/run_reject_tests.sh）。期待は reject（型エラーで止まる）か warn（警告は出るが通る。期限だけ）である。

テスト	見るもの	OCaml	Py-I	JS-I
r1	reply の被覆漏れ	○	○	○
r2	reply の線形性違反	○	○	○
r3	期限のない now（警告）	○	○	○
r4	効果の宣言漏れ（mut）	○	○	○
r5	効果（future+await 経由）	○	○	○
r6	アクター型の不一致	○	○	×
r7	効果（now 経由。r5 の対照）	○	○	○
r8	1 文字のクラス名	○	×	×
r9	未宣言の名前への代入	○	○	○
r10	存在しないメソッドへの send	○	○	×
r11	引数の数が合わない	○	○	×
r12	引数の型が合わない	○	○	×
r13	reply の型が宣言と違う	○	○	×
r14	future でないものを await	○	×	×
r15	期限が 0 ミリ秒	○	○	○

作業前は 8 本 24 枠のうち **11 枠**が期待と食い違っていた（OCaml 3、Py-I 3、JS-I 5）。その 8 本を直したうえで、さらに探りを入れて 7 本を足した。現在は 15 本 45 枠のうち **9 枠**である。OCaml 版は **15 本すべて**を弾く。残りは Py-I が 2（r8・r14）、JS-I が 7 である。

■探りで新たに出た穴。 r9 が三実装すべてに共通していた唯一の穴である。**未宣言の名前への代入が黙って通り、その場で新しい変数ができていた**。読み出しは unbound variable で弾いているのに、代入だけが素通りしていた。

```
class C {
  var count = 0;
  method f(x) : int { cout = count + 1; reply(count); }   // count の打ち間違い
}
```

count は永久に 0 のままで、何のエラーも出ない。三実装とも弾くようにした（AIOS_LAX_ASSIGN=1 で従来の暗黙宣言に戻せる）。543 本を通した結果、**影響を受けた既存ファイルは 0 本**であった。暗黙宣言に頼っているコードは一つも無かったことになる。

残る r10–r15 は「OCaml 版にはあるが他の実装に無い検査」で、穴というより追随の遅れである。今回 Py-I にメソッドの存在・引数の数・引数の型・reply の型・期限の値の検査を、JS-I に期限の値の検査を足した。

7.4 三実装を並べて初めて見えたもの

この集合を作る過程で、**出力の比較では絶対に出てこない**食い違いが五つ出た。いずれも「検査したのに何も出ない」類である。

1. **効果伝播の穴は OCaml 版だけのものだった**。Py-I は future+await でも正しく効果を引き継いでいた。正であるはずの実装が唯一間違えていた例である。

2. **JS-I は効果を集めていたが、宣言と照合していなかった。**呼び出しの辺も不動点も持っているのに、その結果を `!{...}` と突き合わせる箇所がどこにも無い。さらに効果の収集がクラス表の登録に依存しており、単体で走らせるとフィールド代入が一件も `mut` にならなかった。
3. **JS-I はメッセージ送信と `reply` を `mut` と数えていた。**仕様では `mut` は「自分の状態を変える」であり、フィールド代入と `become` だけが生む。照合を入れた瞬間、正しいプログラムが軒並み落ちてこれに気づいた。
4. **Py-I はトップレベルの文を毎回空の環境で検査していた。**`var u = new Use(); send u.f(b);` の `u` が未知になり、トップレベルの呼び出しは**引数がひとつも照合されていなかった。**

`send` の効果の扱いが三者三様だった。ガイドの `g6` は「`send` は待たないので引き継がない」と明記しているのに、Py-I と JS-I は引き継いでいた。そのため**仕様どおりに書いたプログラムが二つの実装で落ちる。**OCaml 版に合わせて直した。判断の材料として、逆に OCaml 版で `send` も引き継ぐようにして 543 本を通してみたところ、落ちたのは**ガイドの `g6` ただ一本**で、それがまさにこの規律を説明している例題であった。

加えて、組込みの効果表そのものが揃っていなかった。`ai_call` は OCaml 版と Py-I が `{ai, net}` を与えるのに対し、JS-I は `{ai}` だけで、「AI を使うが機外へは出ない」オンデバイス推論と区別がつかなくなっていた。これも揃えた。

所見 8. 残る 3 枠のうち 2 枠は `r8` である。Py-I は **1 文字の大文字を型変数として読む**（`\b[A-Z]\b`）。そのため `class A` を作って `method f(x: A)` と書くと、`A` はクラスではなく総称の枠になり、何を渡しても通る。OCaml 版はクラス名として読む。同じ誤りが**クラス名の長さによって**捕まったり捕まらなかったりする。これは実装の不具合というより、言語仕様が「型変数を書けるのか、書けるならどう綴るのか」を決めていないことの現れである。

7.5 その他

- メソッド内の `return` が JS-I に無い（Py-I は受ける）。
- `typeof` の出力書式が三実装で揃っていない。JS-I には存在しない。
- 型注釈つき変数宣言 `var x : T = e` が未実装。
- OCaml 版 REPL の出力取りこぼし（§6 で定量化）。
- JS-I にメソッドの存在・引数・アクター型の照合が無い（`r6`・`r10`–`r13`）。送信の宛先クラスを最善努力で解決しているだけで、引数を名前で照合する経路を持たない。
- Py-I に「future でないものを `await` している」の検査が無い（`r14`）。
- 型変数の綴りが言語仕様に無い（`r8`）。

8 小林研究をどう取り込むか

§2.2 で挙げた道具箱は、AIPL が残した課題にそのまま噛み合う。対応を表にする。

AIPL の課題	小林の道具	得られるもの
返信先を一級にしたいが、「ちょうど一度」を守れない（§3.10）	線形型 [9]	返信先を線形な値にする。委譲が書けて、なお二重返信・返信忘れが型で止まる
進行定理の第三の枝（期限の無い場合の待ちの循環）	デッドロック自由な型システム [10, 11, 12]	期限に頼らずに枝を消す。期限は工学的な保険として残せる
二つのプロファイル（エージェント／実時間）を別々の規則で書いている	π 計算の汎用型システム [13]	一つの枠組みからプロファイルを導く。規則の重複が消える
効果が順序を持たない集合である	資源使用解析 [14]	「開いたら閉じる」「取得したら解放する」を効果の水準で書ける
`where` ガードを戻すと検証が要る（§3.10）	高階モデル検査 [15, 16]、述語抽象と CEGAR	検証器を自作せずに借りる
期限の値が妥当かを言えない	精製型・サイズ型 [17]	応答時間の上界を型で述べる

すべてを取り込むことは可能でも望ましくもない。別稿 [25] で代償を含めて検討したが、本稿の立場から言えば優先順位は明確である。

1. **線形型による返信先の一級化**。ABCL/1 から落とした三つのうち、唯一「型の問題」であるものが解ける。委譲が書けるようになる効果も大きい。
2. **効果に順序を入れる**。§7 の一つ目の穴を直したうえで、効果を集合から順序つきの構造にすれば、デバイスの獲得と解放のような性質が書ける。OS を記述する目的 [27] に直結する。
3. **デッドロック自由な型システム**。現在は期限で回避しているが、期限は「待ちが終わる」ことしか保証しない。設計として循環を排除できるほうが強い。

9 関連研究

ABCL 系については §2 で述べた。ABCL/1[1, 2] が計算モデルの出発点であり、ABCL/f[3] は future を中核に据えた型付きの先行例、ABCL/R[4] は反射的な拡張である。ABCL カーネル言語の意味論 [5] は本プロジェクトでも形式化の対象としている。

型付き並行言語という枠では、セッション型が「通信の順序を型で書く」立場から近い位置にある。AIPL は現在、実行時のセッションプロトコル検査を持つが、静的なセッション型は持たない。効果システムの側では、効果を集合として扱う素朴な形を採っており、順序や資源の獲得・解放を表現できない。この二点はいずれも §8 の方向で改善しうる。

注意 9. 名称について一点断っておく。本プロジェクトの過去のレポートに、GA と複数の言語モデルを組み合わせた別のシステムを “AIPL (AI Programming Loop)” と呼んでいるものがある [29]。本稿の AIPL（Actor based Intelligent Parallel Language）とは別物である。

10　おわりに

AIPL は ABCL/1 の三種のメッセージ送信を継承し、そこに型・効果・期限という三つの静的な仕組みを載せた並行アクター言語である。本稿はその設計と実現を一つにまとめ、実測にもとづいて評価した。

得られたことを三点にまとめる。第一に、**三つの仕組みは互いに噛み合う**。効果と期限を組み合わせると、「実時間で動くアクターが機外のモデルを同期で待つ」という設計上の誤りが型検査で止まる。第二に、**期限を構文に義務づけると進行定理の枝が一本減る**。その結果、デッドロック自由が系として言える。ただしこれは紙の上の結果であって、機械検証はまだ済んでいない（残る作業は §4.4 の三点）。代償は停止性が言えないことと、デッドロックが消えるのではなく有限時間の失敗に変わることである。第三に、**形式化は実装の穴を見つける**。本稿で報告した二つの穴——効果伝播の抜け道とアクター型の非区別——は、どちらも正しいプログラムでは表に出ず、仕様を書き下す過程で初めて露見した。

残された課題も明確である。実装側では二つの穴の修正、設計側では線形型による返信先の一級化と、効果への順序の導入である。これらは ABCL/1 から落としたものを取り戻す作業でもあり、小林の系譜が示した道具でそれが可能になる、という見通しを本稿は述べた。

再現方法

本稿の実測値は次で再現できる。

```
# 三実装の一致
sh ~/aios/abclcp/tools/run_all_impls.sh

# 単一サンプルの反復（OCaml 版）
cd ~/aios/abclcp
printf 'load %s\ncompile\n' docs/samples/guide/g9_actor_arg.aipl > /tmp/r.repl
./src/abclrepl_thread -q -f /tmp/r.repl

# 証明の確認
cd ~/aios/abclcp/coq
coqc -Q . "" AIPLSoundness.v
echo 'Require Import AIPLSoundness. Print Assumptions type_safety.' > /tmp/chk.v
coqc -I . -Q . "" /tmp/chk.v
```

参考文献

[1] Akinori Yonezawa, Jean-Pierre Briot, Etsuya Shibayama. Object-Oriented Concurrent Programming in ABCL/1. *OOPSLA 1986*, pp. 258–268.

[2] Akinori Yonezawa (ed.). *ABCL: An Object-Oriented Concurrent System*. MIT Press, 1990.

[3] Kenjiro Taura, Satoshi Matsuoka, Akinori Yonezawa. ABCL/f: A Future-Based Polymorphic Typed Concurrent Object-Oriented Language. 1994.

[4] Takuo Watanabe, Akinori Yonezawa. Reflection in an Object-Oriented Concurrent Language (ABCL/R). *OOPSLA 1988*.

[5] Etsuya Shibayama. An ABCL Kernel Language and Its Semantics.

[6] Naoki Kobayashi, Akinori Yonezawa. Asynchronous Communication Model Based on Linear Logic. 1992.

[7] Naoki Kobayashi, Akinori Yonezawa. ACL — A Concurrent Linear Logic Programming Paradigm. 1993.

[8] Naoki Kobayashi, Akinori Yonezawa. Type-Theoretic Foundations for Concurrent Object-Oriented Programming. *OOPSLA 1994*.

[9] Naoki Kobayashi, Benjamin C. Pierce, David N. Turner. Linearity and the Pi-Calculus. *POPL 1996*.

[10] Naoki Kobayashi. A Partially Deadlock-Free Typed Process Calculus. 1997.

[11] Naoki Kobayashi. An Implicitly-Typed Deadlock-Free Process Calculus. 2000.

[12] Naoki Kobayashi. A New Type System for Deadlock-Free Processes. *CONCUR 2006*.

[13] Naoki Kobayashi. A Generic Type System for the Pi-Calculus. *POPL 2001*.

[14] Naoki Kobayashi et al. Resource Usage Analysis. 2001/2002/2005.

[15] Naoki Kobayashi. Types and Higher-Order Recursion Schemes for Verification of Higher-Order Programs. 2009.

[16] Naoki Kobayashi, C.-H. Luke Ong. A Type System Equivalent to the Modal Mu-Calculus Model Checking of Higher-Order Recursion Schemes. 2009.

[17] Naoki Kobayashi et al. Sized Types and Parallel Complexity.

[18] 児玉靖司. AIPL（ABCLc+）と ABCL/1 — 機能とモデルの観点からの比較. `docs/aipl_vs_abcl1_ja.pdf`, 2026 年 7 月 30 日.

[19] 児玉靖司. AIPL の言語仕様の検討 — ABCL/1 の機構の再導入と、分散 AI エージェント／実時間 OS という目的から. `docs/aipl_abcl1_features_ja.pdf`, 2026 年 7 月 30 日.

[20] 児玉靖司. AIPL における型の健全性と型の安全性 — 証明が成立する範囲 AIPL^{-} の定式化と機械検証. `docs/aipl_soundness_ja.pdf`, 2026 年 7 月 28 日.

[21] 児玉靖司. AIPL の型健全性と型安全性（第 2 版）— 現行実装から証明可能な範囲 AIPL^{-2} を切り出し、効果と期限つきで証明する. `docs/aipl_soundness2_ja.pdf`, 2026 年 7 月 30 日.

[22] 児玉靖司. AIPL におけるメソッド戻り値型の推論 — `reply` の引数から型は決まるか. `docs/aipl_reply_inference_ja.pdf`, 2026 年 7 月 29 日.

[23] 児玉靖司. AIPL / ABCLc+ ユーザーズガイド — 文法・使い方・サンプルプログラム. `docs/aipl_guide_ja.pdf`, 2026 年 7 月 30 日.

[24] 児玉靖司. AIPL 3 実装の同期 — 拡張子の統一、仕様追随、そして調べる過程で出た 6 つの不具合. `docs/aipl_three_impl_parity_ja.pdf`, 2026 年 8 月 1 日.

[25] 児玉靖司. 小林研究の成果を AIPL に取り入れる場合の言語仕様 — 可能性、メリット、デメリット. `docs/aipl_kobayashi_synthesis_ja.pdf`, 2026 年 7 月 30 日.

[26] 児玉靖司. 移動エージェントによる避難支援 MANET の AIPL×soft-Xinu 実装と評価. 2026 年 7 月.

[27] 児玉靖司. actor-first 記述言語 AIPL による小型 OS の記述. 2026 年 5 月.

[28] 児玉靖司. AIPL/AICE 汎用進化的 Actor アーキテクチャ探索フレームワークとその評価. 2026 年 7 月.

[29] 児玉靖司. 複数 LLM エージェントの協調による進化計算. 2026 年 5 月.（本稿の AIPL とは別システム）